



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

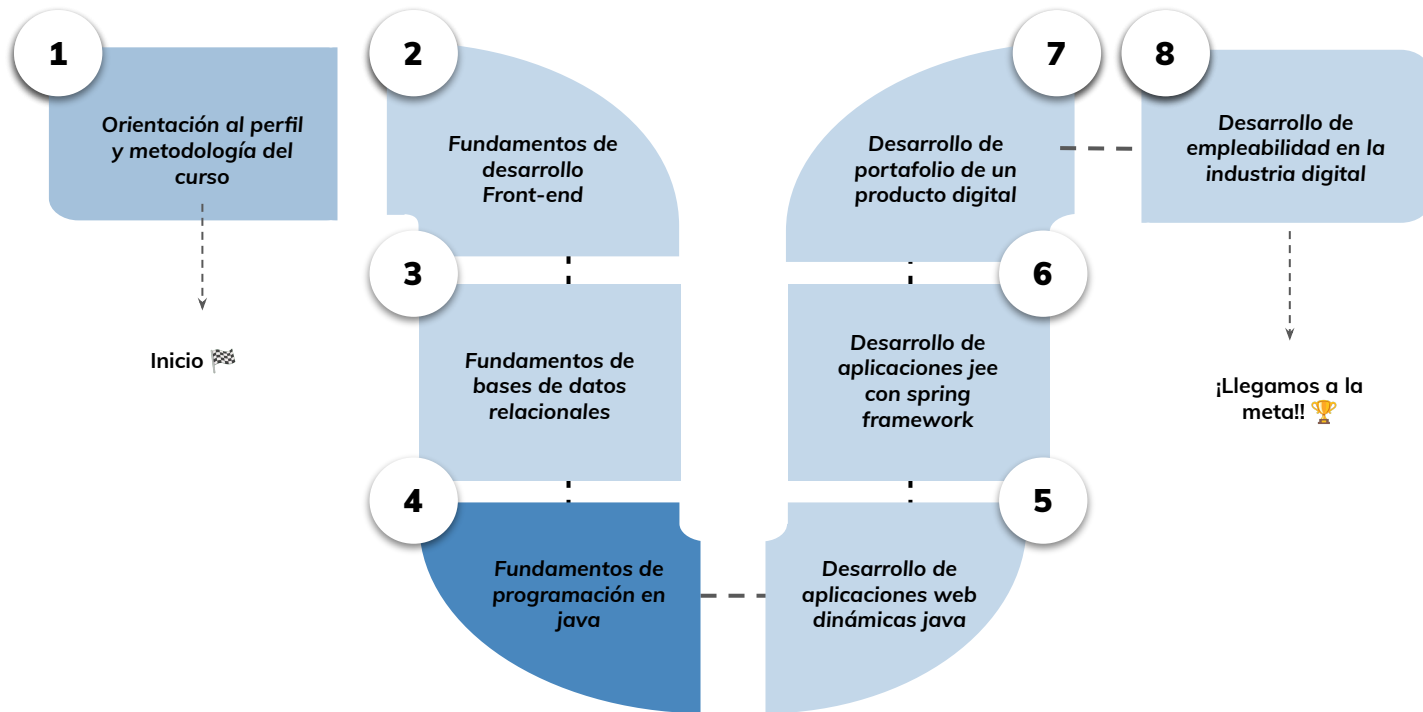


› El paradigma de orientación a objetos algoritmos- Parte 2

Aprendizaje Esperado 5: Utilizar elementos de la programación orientada a objetos para la implementación de una pieza de software que da solución a un problema de baja complejidad.

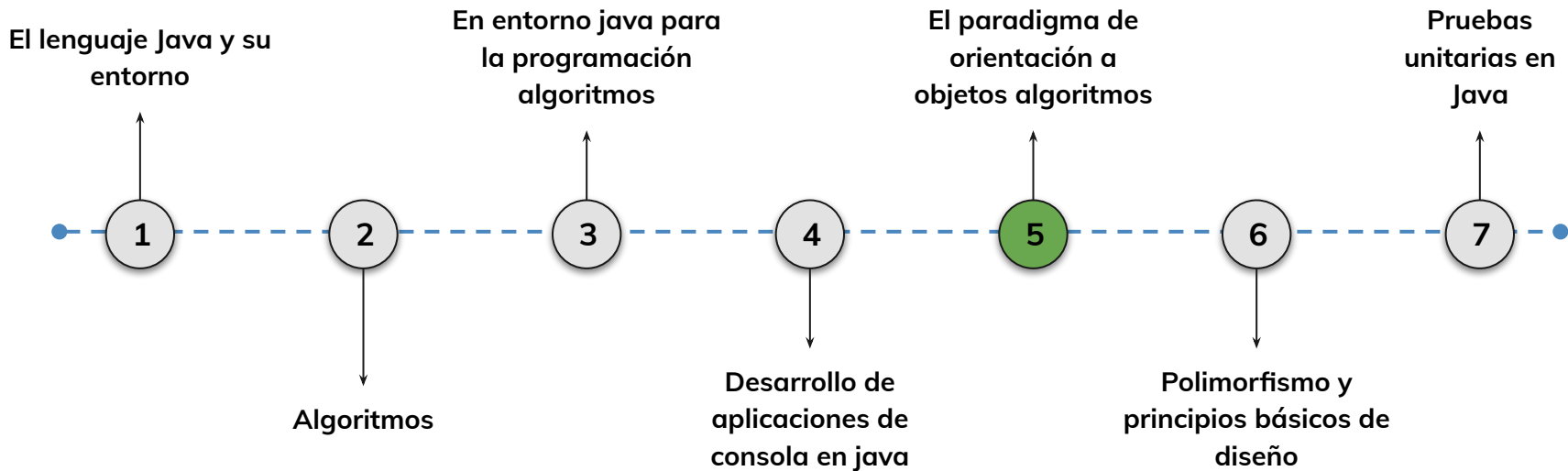
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Aplicación de
comportamiento orientado a
objetos en algoritmos

En esta clase profundizaremos en la interacción entre objetos en Java, explorando el uso de métodos con parámetros, retorno de valores, composición y relaciones entre objetos. A través de ejercicios prácticos comprenderemos cómo se comunican las clases para construir soluciones más estructuradas y modulares.

Accesadores y Mutadores

Métodos que
reciben objetos
como parámetros

Métodos que
retornan objetos

Colaboración
entre objetos

Composición y
asociación

Interacción entre
instancias y diseño
colaborativo



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender cómo aplicar los métodos de acceso y modificación de objetos.
- Comprender el concepto de relación entre clases
- Reconocer las relaciones de agregación y composición
- Comprender la representación de clases a través de diagramas de flujo

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos la importancia de la POO
- Conocimos la implementación de Clases y el estado y comportamiento de los Objetos
- Comprendimos el concepto e implementación de los métodos constructores
- Reconocimos la utilidad e implementación de los modificadores de acceso

➤ Accesadores y Mutadores

< > Accesadores

¿Qué son los getters?:

Los accesadores o métodos **get** se utilizan para **consultar el estado de un objeto** .

Un método get se declara **public** y a continuación se indica el tipo de dato que devuelve.

El método no recibe parámetros y en el cuerpo del método se utiliza **return** para devolver el valor correspondiente al atributo requerido.

```
public String getNombre() {  
    return nombre;  
}
```



< > Mutadores

¿Qué son los setters?:

Los mutadores o métodos **set** se utilizan para **modificar el estado de un objeto** .

Un método **set** se declara **public** y devuelve **void**.

La lista de **parámetros** incluye **el tipo y el valor a modificar** y en el cuerpo del método se asigna el parámetro de la declaración al atributo del objeto.

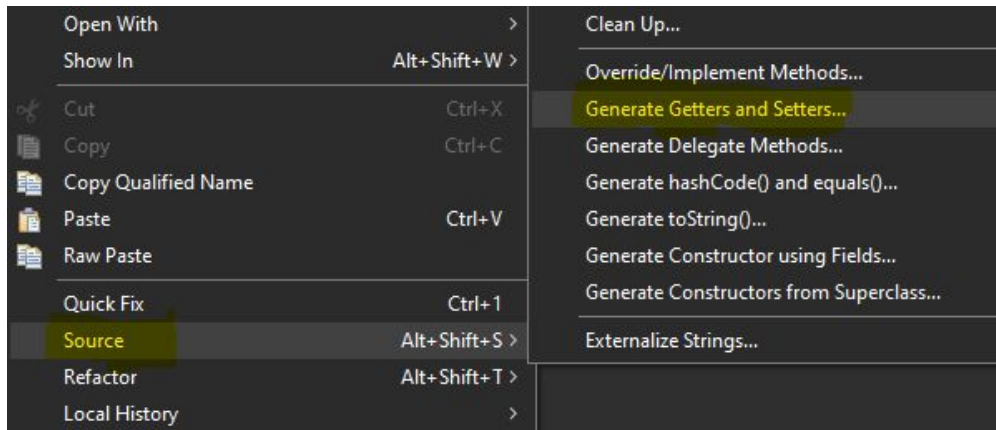
```
public void setNombre(String nombre) {  
    this.nombre= nombre;  
}
```



< > Accesadores y Mutadores

Los getters y setters, junto con los modificadores de acceso, permiten establecer un nivel adecuado de **control y seguridad en el acceso y modificación de los atributos** de una clase. Ayudan a mantener la coherencia y consistencia de los datos, además de proporcionar una interfaz más clara y segura para interactuar con los objetos de una clase.

Eclipse permite generar los get y set de cada atributo de manera automatizada haciendo **clic derecho** sobre el editor de texto, seleccionando **Source** -> **Generate Getters and Setters**.



LIVE CODING

Ejemplo en vivo

Manipulando atributos:

En este ejercicio vamos a manipular objetos y atributos para analizar su comportamiento mediante los métodos de consulta y modificación.

1. *En la clase llamada Cuenta (bancaria) que posee como mínimo los atributos: numeroCuenta , el nombre del cliente y el saldo actual.*

LIVE CODING

Ejemplo en vivo

Las operaciones asociadas a dicha clase son:

1. **Constructor** por defecto y constructor con saldo, número de cuenta y nombre.
2. Agregar los métodos **getters** y **setters** correspondientes
3. Método para **crear** un objeto Cuenta, pidiéndole los datos al usuario.

Luego, vamos a realizar pruebas para acceder y modificar los datos de una cuenta y ver así el comportamiento de los métodos accesadores y mutadores



Ejercicio N° 1

< > Cafetera

¡Vamos a programar una cafetera! 🙌

Es importante tener el control de la creación de objetos en nuestro programa, así como supervisar y asegurarnos de siempre estar trabajando con el mismo objeto. Es por esto que vamos a practicar la manipulación de atributos en el siguiente ejercicio:

Consigna: 📝

Desarrolle una clase **Cafetera** con los atributos **capacidadMaxima** (la cantidad máxima de café que puede contener la cafetera) y **cantidadActual** (la cantidad actual de café que hay en la cafetera).

Tiempo 🕒: 40 minutos



< > Cafetera

Métodos solicitados:

1. **Constructor predeterminado** o vacío.
2. **Constructor** con la capacidad máxima y la cantidad actual.
3. Métodos **getters** y **setters**.
4. Método **llenarCafetera** (): su función es hacer que la cantidad actual sea igual a la capacidad máxima.
5. Método **servirTaza** (int): se pide el tamaño de una taza vacía, el método recibe el tamaño de la taza y simula la acción de servir la taza con la capacidad indicada. Si la cantidad actual de café “no alcanza” para llenar la taza, se sirve lo que reste en la máquina. El método le informará al usuario si se llenó o no la taza, y en caso de no haberse llenado informará en cuanto quedó la taza.
6. Método **vaciarCafetera** (): define la cantidad de café actual en cero.
7. Método **agregarCafe** (int): se le pide al usuario una cantidad de café, el método lo recibe y se añade a la cafetera la cantidad de café indicada.



➤ Colaboración entre objetos

< > Colaboración entre objetos

¿Qué es la relación entre clases?:

En la programación orientada a objetos, las clases se relacionan entre sí de diferentes formas para modelar situaciones de la vida real. Existen dos tipos principales de relaciones entre clases: Agregación y Composición.

Estas relaciones permiten modelar escenarios complejos del mundo real de forma intuitiva y fácil de entender. Ayudan a reutilizar código y aplicar los principios de encapsulamiento y ocultación de información.



< > Agregación

¿Qué es la relación de agregación?:

La agregación es una relación donde un objeto **"tiene"** a otros objetos. Por ejemplo, un automóvil tiene ruedas, un edificio tiene ventanas.

En la agregación, los objetos relacionados **pueden existir de forma independiente** . Si eliminamos el objeto contenedor, los objetos contenidos pueden seguir existiendo.



< > Composición

¿Qué es la relación de composición?:

La composición es una relación donde un objeto **"está formado por"** otros objetos. Por ejemplo, una casa está formada por paredes, puertas y ventanas. Un empleado está formado por información personal.

En la composición, los objetos relacionados dependen totalmente del objeto contenedor. **Si eliminamos el objeto contenedor, los objetos contenidos dejan de existir.**





LIVE CODING

Ejemplo en vivo

Relacionando objetos:

En este ejercicio vamos a seguir trabajando sobre el proyecto de BilleteraVirtual que venimos desarrollando en clase. Es importante tener en cuenta qué tipo de relación vamos a forjar entre diferentes clases.

Realizar una clase llamada Cliente que debe tener los atributos: ID (entero largo), nombre (String), cuenta (Cuenta).

Definir qué tipo de relación existe entre las clases.

LIVE CODING

Ejemplo en vivo

Las operaciones asociadas a dicha clase son:

1. Agregar los métodos **getters** y **setters** correspondientes
2. Método para **crear** un objeto Cliente, pidiéndole los datos al usuario.
3. Método para **depositar()** en una Cuenta: el método recibe una cantidad de dinero a ingresar y ese monto debe sumarse al saldo actual.
4. Método **retirar()** para la clase Cuenta que permita retirar fondos de la cuenta, restando el importe del saldo actual.
5. Método **consultarDatos()**: permitirá mostrar todos los datos de la cuenta y del cliente vinculado.

Tiempo: 40 minutos



Ejercicio N° 1

< > Mascotas

¡Vamos a adoptar mascotas! 🙌

En este ejercicio vamos a realizar un programa para que una Persona pueda adoptar un Perro. El programa debe contar de dos clases:

- **Perro**, que tendrá como atributos: nombre, raza, edad y tamaño.
- **Persona** con atributos: nombre, apellido, edad, RUT y Perro.

Además, se debe crear en el main **dos Personas y dos Perros** .

Es necesario asignarle a cada Persona un Perro y por último, mostrar desde la clase Persona, la información de los objetos Perro y Persona.

Tiempo 🕒: 40 minutos



➤ Diagrama de Clases

< > Diagrama de Clases

¿Qué es y para qué se utiliza?

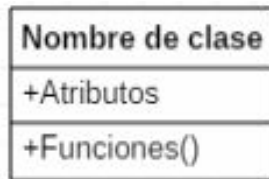
El diagrama de clases es un diagrama puramente orientado al modelo de programación orientado a objetos, ya que **define las clases que se utilizarán** cuando se pase a la fase de construcción **y la manera en que se relacionan** las mismas. Su utilidad es la representación de datos y su interacción. Este diagrama **muestra el modelo lógico de los datos de un sistema.**



< > Diagrama de Clases

Cada clase está representada por un rectángulo que tiene una subdivisión de tres zonas: **nombre** , **atributos** y **métodos** .

- La primera de las zonas se utiliza para el nombre de la clase. En caso de que la clase sea abstracta se utilizará su nombre en cursiva.
- La segunda de las zonas se utiliza para escribir los atributos de la clase, uno por línea.
- La última de las zonas incluye cada uno de los métodos que ofrece la clase.

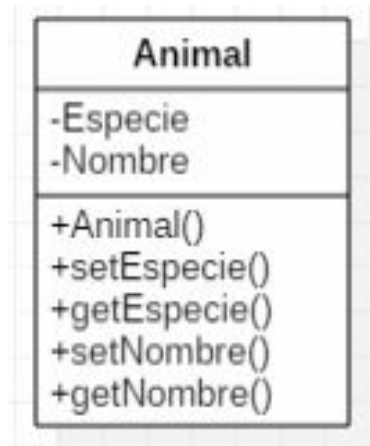


< > Diagrama de Clases

Tanto los atributos como los métodos incluyen al principio de su descripción la **visibilidad** que tendrá. Esta visibilidad se identifica escribiendo un símbolo y podrá ser:

- **(+) public** : Representa que se puede acceder al atributo o función desde cualquier lugar de la aplicación.
- **(-) private** : Representa que se puede acceder al atributo o función únicamente desde la misma clase.
- **(#) protected** : Representa que el atributo o función puede ser accedida únicamente desde la misma clase o desde las clases que hereden de ella (clases derivadas).

Estos tres tipos de visibilidad son los más comunes. No obstante, pueden incluirse otros en base al lenguaje de programación que se esté usando (no es muy común). Por ejemplo: (/) Derivado o (~) Paquete.

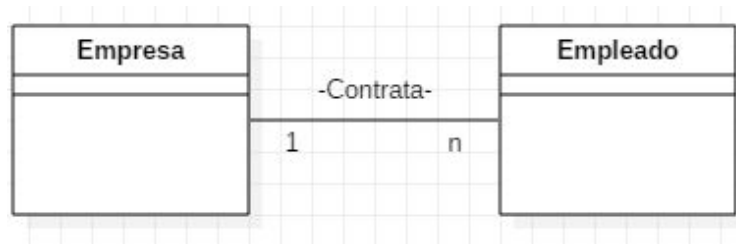


< > Diagrama de Clases

Relaciones :

Las relaciones en el diagrama de clases tienen varias propiedades, que dependiendo la profundidad que se quiera dar al diagrama se representarán o no. Estas propiedades son las siguientes:

- **Multiplicidad** . Es decir, el número de elementos de una clase que participan en una relación. Se puede indicar un número, un rango... Se utiliza **n** o ***** para identificar un número cualquiera.
- **Nombre de la asociación** . En ocasiones se escriba una indicación de la asociación que ayuda a entender la relación que tienen dos clases. Suelen utilizarse verbos como por ejemplo: «Una empresa **contrata** a n empleados»



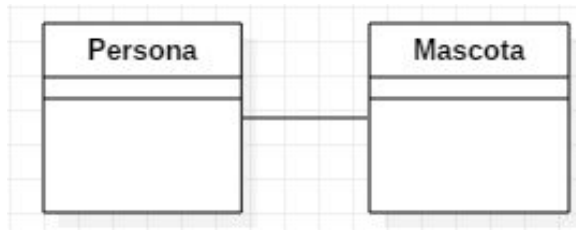
< > Diagrama de Clases

Relación de Asociación :

Este tipo de relación es el más común y se utiliza para representar dependencia semántica. Se representa con **una simple línea continua** que une las clases que están incluidas en la asociación.

Un ejemplo de asociación podría ser:

«**Una** mascota **pertenece** a **una** persona».



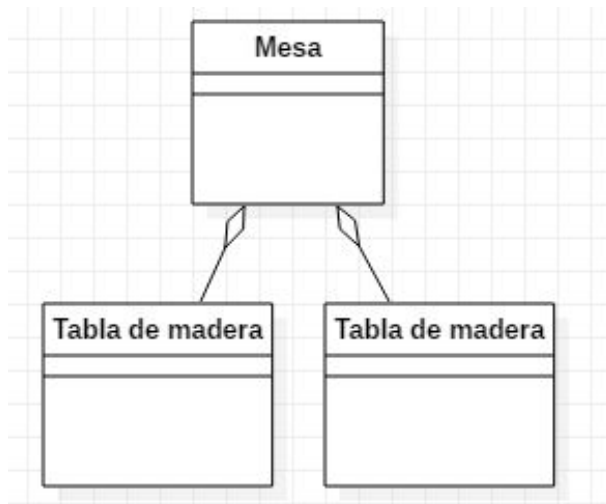
< > Diagrama de Clases

Relación de Agregación :

Es una representación jerárquica que indica a un objeto y las partes que componen ese objeto. Es decir, **representa relaciones en las que un objeto es parte de otro** , pero aun así debe tener existencia en sí mismo .

Se representa con **una línea que tiene un rombo** en la parte de la clase que es una agregación de la otra clase (es decir, en la clase que contiene las otras).

Un ejemplo de esta relación podría ser: «Las mesas **están formadas por** tablas de madera y tornillos o, dicho de otra manera, los tornillos y las tablas **forman parte de** una mesa».



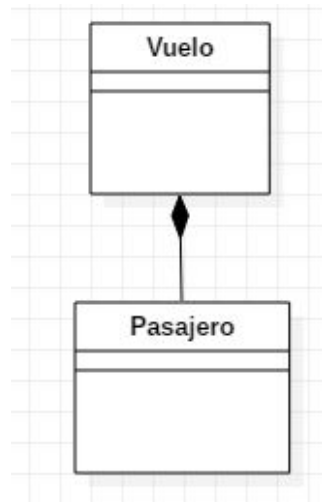
< > Diagrama de Clases

Relación de Composición :

En este caso, **los elementos que forman parte no tienen sentido de existencia cuando el primero no existe** . Es decir, cuando el elemento que contiene los otros desaparece, deben desaparecer todos ya que no tienen sentido por sí mismos sino que dependen del elemento que componen.

Se representa con una **línea continua con un rombo relleno** en la clase que es compuesta.

Un ejemplo de esta relación sería: «Un vuelo de una compañía aérea **está compuesto por** pasajeros, qué es lo mismo que decir que un pasajero **está asignado a** un vuelo»



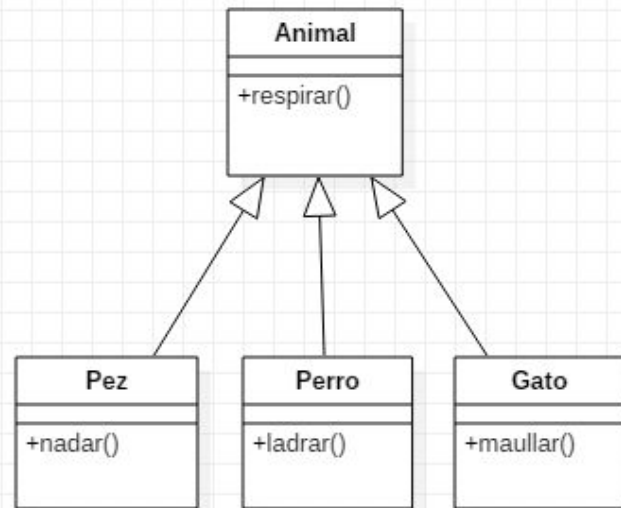
< > Diagrama de Clases

Relación de Herencia :

Este tipo de relaciones **permiten que una clase** (clase hija o subclase) **reciba los atributos y métodos de otra clase** (clase padre o superclase). Estos atributos y métodos recibidos se suman a los que la clase tiene por sí misma. Se utiliza en relaciones **«es un»**.

Un ejemplo de esta relación podría ser la siguiente: Un pez, un perro y un gato **son** animales.

En este ejemplo, las tres clases (Pez, Perro, Gato) podrán utilizar el método **respirar**, ya que lo heredan de la clase animal, pero solamente la clase Pez podrá nadar, la clase Perro ladrar y la clase Gato maullar.



LIVE CODING

Ejemplo en vivo

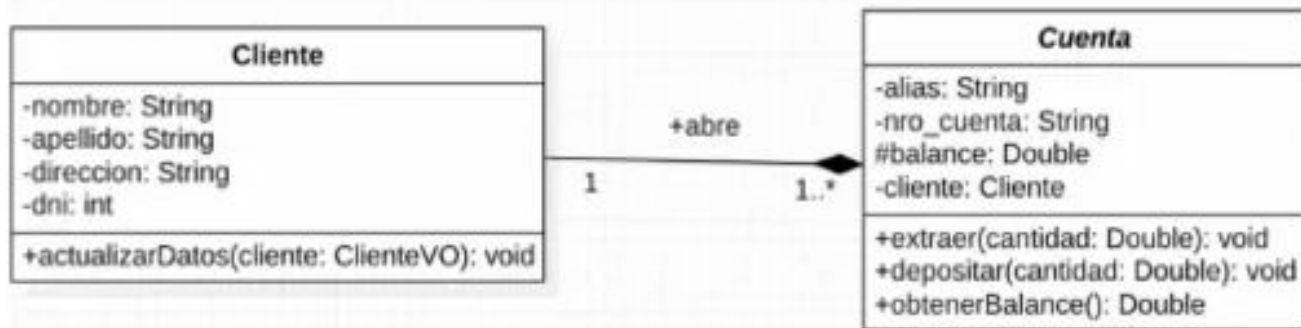
Creando programas a partir del diagrama:

En programación es habitual crear primero la estructura del programa que resuelve una necesidad determinada en un diagrama de clases y luego codificar. Es por esto que en este primer ejemplo realizaremos el siguiente ejercicio.

1. Crear un programa que respete el diagrama de clases dado.

LIVE CODING

Ejemplo en vivo



Tiempo: 30 minutos



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Reconocimos la importancia del encapsulamiento de información a través de la implementación de métodos accedadores y mutadores (get y set)
- ✓ Reconocimos las relaciones de agregación y composición.
- ✓ Comprendimos la implementación de las relaciones entre clases
- ✓ Comprendimos la utilidad de los diagramas de clase para la consecuente codificación



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

